ARCHITECTURE SPECIFICATION

Personal Nutrition Tracker — Tech Stack Blueprint

A comprehensive overview of purpose, limitations, local-first offline storage, and learning pathways for a custom single-user mobile app.

1. Executive Tech Stack Summary

This application is engineered as a lightweight, monolithic, offline-first mobile app for Android. By pairing React Native (Expo) with local SQLite persistence and free external APIs, the architecture provides zero-latency logging without recurring subscription costs.

Layer	Selected Technology	Core Responsibility
Mobile Framework	React Native (Expo Go)	Cross-platform UI execution and fast mobile deployment.
Routing	Expo Router	File-based navigation across log, search, and custom screens.
Local Database	expo-sqlite + Drizzle ORM	Offline storage for daily nutrition logs and cached food items.
Network Listener	@react-native-community/netinfo	Detects connection state changes to trigger offline sync tasks.
External APIs	USDA FoodData + FatSecret API	Querying whole ingredients, commercial brands, and packaged goods.

2. Technology Breakdown: Purpose, Limitations & Learning Curve

React Native & Expo Framework

Purpose: Serves as the core runtime and UI framework. Expo simplifies native device access, build pipelines, and phone testing via Expo Go.

- **Limitations:** Slightly larger application bundle size compared to pure native Kotlin; limited access to deeply custom native C++ modules (unneeded for this scope).
- **Learning Curve:** *Low to Moderate.* If familiar with JavaScript/React fundamentals, building functional screens takes only a few days. Focus on React hooks (`useState`, `useEffect`) and basic Layout Flexbox.

Local Persistence: expo-sqlite & Drizzle ORM

Purpose: Provides an offline-first relational database embedded on the phone. Stores user logs, custom recipes, and local API caches so the app functions seamlessly without internet connectivity.

- **Limitations:** Local SQLite is device-bound; data must be manually backed up or synced to a remote host if migrating phones.
- **Learning Curve:** *Moderate.* Requires understanding basic SQL schemas (tables, primary keys, `is_synced` flags). Using Drizzle ORM simplifies raw SQL string writing into standard JavaScript syntax.

Nutrition APIs: USDA & FatSecret Platform

Purpose: USDA provides unthrottled access to raw whole foods and micro/macro profiles. FatSecret acts as a fallback for commercial branded items and restaurant lookups.

- **Limitations:** FatSecret basic tier enforces daily request quotas. Caching frequent items locally is required to stay within limits.
- **Learning Curve:** *Low.* Basic REST API fetching using standard `fetch` or `axios` to parse JSON payloads.

3. Step-by-Step Learning Roadmap

1. **Phase 1: Fundamentals (Days 1–3):** Set up Expo CLI. Build simple React Native layout components (Text, View, TextInput, Pressable) with flexbox alignment.
2. **Phase 2: Local Database Setup (Days 4–6):** Initialize `expo-sqlite`. Create tables for `food_logs` and `cached_foods`. Implement local CRUD (Create, Read, Update, Delete) operations.
3. **Phase 3: External API Integration (Days 7–9):** Integrate USDA API calls for generic food queries. Parse macro responses into local database format. Add FatSecret API keys for branded items.
4. **Phase 4: Offline-First Synchronization (Days 10–12):** Implement `@react-native-community/netinfo`. Configure sync state handling (`is_synced = 0`) to write locally when offline and background-sync when online.